

Static Analysis Report

OWASP UnCrackable Level 2

Analyst : Omayma EL-Yamani

Date : April 26, 2026

A) General Information

APK File	UnCrackable-Level2.apk
Package Name	sg.vantagepoint.uncrackable2
Native Library	libfoo.so
Source	OWASP MAS Crackmes
Tools Used	JADX GUI, Ghidra, Python

B) Executive Summary

This static analysis revealed **3 critical vulnerabilities** in the OWASP UnCrackable Level 2 application. The main concerns are **secret stored in native library without protection**, **weak anti-debugging mechanisms**, and **insufficient root detection**. The overall risk level is **Critical**.

Priority Actions:

1. Never store secrets in client-side code, even in native libraries
2. Move validation logic to server-side
3. Implement SafetyNet Attestation for root detection

C) Detailed Findings

Finding #1: Secret Stored in Hexadecimal within Native Library

Severity	Critical
Description	The secret is stored as reversed hexadecimal ASCII within the native library <code>libfoo.so</code> . The value <code>6873696620656874206c6c6120726f6620736b6e616854</code> is compared to user input via <code>strncmp</code> . After hexadecimal conversion and string reversal, the secret "Thanks for all the fish" was revealed.
Location	<code>libfoo.so</code> — function <code>Java_sg_vantagepoint_uncrackable2_CodeCheck_bar</code>
Impact	Any attacker can extract the secret through static analysis of native code using Ghidra, completely bypassing the application's protection.
Remediation	Never store secrets in plain text within native code. Use server-side validation or implement strong encryption with dynamically derived keys.

Finding #2: Weak Anti-Debugging Protection

Severity	Critical
Description	The native method <code>init()</code> implements anti-debugging protection based on <code>fork/ptrace</code> . Additionally, <code>MainActivity</code> uses <code>Debug.isDebuggerConnected()</code> in a loop and checks if the application is debuggable. These protections are easily bypassed using Frida or by patching the APK.
Location	<code>libfoo.so</code> — <code>init()</code> function and <code>MainActivity.onCreate()</code>
Impact	Anti-debugging mechanisms are ineffective, allowing attackers to use dynamic debugging tools to analyze the application at runtime.
Remediation	Implement more advanced anti-debugging techniques such as hardware breakpoint detection, code integrity checks, and Frida hook verification.

Finding #3: Insufficient Root Detection

Severity	Medium
Description	The <code>MainActivity</code> class contains three root detection methods (<code>b.a()</code> , <code>b.b()</code> , <code>b.c()</code>) that check for su binaries, busybox, and suspicious system properties. These client-side checks are easily bypassed using Magisk Hide or by patching the APK.
Location	<code>MainActivity.onCreate()</code>
Impact	The application can run on rooted devices, potentially exposing sensitive data and allowing the use of dynamic analysis tools.
Remediation	Implement Google SafetyNet Attestation API for robust device integrity checks. Combine with server-side validation of attestation results.

D) Detailed Analysis Process**Step 1: Java Decompilation with JADX**

1. Opened APK in JADX GUI
2. Identified `MainActivity` class as entry point
3. Discovered `CodeCheck` class with native method `bar()`
4. Located native library loading: `System.loadLibrary("foo")`

Step 2: Native Library Extraction

```
unzip UnCrackable-Level2.apk -d extracted/  
ls -R extracted/lib/  
# libfoo.so found in lib/x86/
```

Step 3: Analysis with Ghidra

1. Imported `libfoo.so` into Ghidra
2. Ran automatic analysis
3. Searched for JNI function: `Java_sg_vantagepoint_uncrackable2_CodeCheck_bar`
4. Analyzed decompiled pseudo-code

5. Identified call to `strcmp`

Step 4: Secret Extraction and Decoding

1. Extracted hexadecimal value: 6873696620656874206c6c6120726f6620736b6e616854
2. Hexadecimal to ASCII conversion using Python:

```
hex_data = "6873696620656874206c6c6120726f6620736b6e616854"
ascii_result = bytes.fromhex(hex_data).decode("ascii")
# Result: "hsif eht lla rof sknahT"
```

3. String reversal:

```
s = "hsif eht lla rof sknahT"
secret = s[::-1]
# Result: "Thanks for all the fish"
```

E) Application Validation Flow

```
User Input
    ↓
MainActivity.verify()
    ↓
CodeCheck.a(String)
    ↓
CodeCheck.bar(byte[]) ← NATIVE (JNI)
    ↓
libfoo.so
    ↓
strcmp(input, reversed_hex_secret)
    ↓
Success: "Thanks for all the fish"
```

F) Permissions and Exported Components

Requested Permissions:

- None

Exported Components:

- MainActivity (Activity) — Exported: Yes

G) Conclusion

Static analysis of the OWASP UnCrackable Level 2 application revealed that the validation secret was stored in a native library as reversed hexadecimal. Although moving to native code makes analysis more complex than simple Java storage, the protection remains insufficient against tools like Ghidra.

The secret **"Thanks for all the fish"** was successfully extracted following a structured methodology:

1. Java code decompilation with JADX

2. Identification of CodeCheck class and native method
 3. Extraction of libfoo.so library
 4. Native code analysis with Ghidra
 5. Location of strcmp call
 6. Extraction of hexadecimal value
 7. Conversion and reversal to obtain the final secret
-

Report for LAB 5 — UnCrackable Level 2